Лабораторная работа № 1
Методика измерения времени выполнения заданного фрагмента программы

Наука начинается с измерения

Цель работы: познакомиться с методами профилирования программ путем измерения длительности выполнения заданных фрагментов кода.

Самым простым способом измерения времени выполнения интересующего фрагмента программы является непосредственное измерение времени, затраченного на выполнение данного участка программы. Для получения текущего времени можно воспользоваться WinAPI-функцией `GetTickCount()` (Windows), возвращающей время с момента старта операционной системы в миллисекундах. В Delphi удобным является использование функции `Now()`, определенной в модуле `SysUtils.pas` и возвращающей значение типа `TDateTime`. Значение типа `TDateTime` может быть преобразовано в значение типа `TTimeStamp` при помощи вызова функции `DateTimeToTimeStamp()`. Основное удобство работы с типом `TTimeStamp` заключается в том, что он представляет собой запись с двумя полями: `Time` – время в миллисекундах с полуночи и `Date` – число дней с начала нашей эры.

Delphi

```
program TimeMeasure;
{$APPTYPE Console}

uses
  SysUtils;

var
  StartTime, EndTime: Integer;

begin
  // Начальное значение времени
  StartTime := DateTimeToTimeStamp(Now()).Time;

  // Анализируемый фрагмент кода
  ...

  // Конечное значение времени
  EndTime := DateTimeToTimeStamp(Now()).Time;

  // Время выполнения
  Writeln('Elapsed time: ',
      EndTime-StartTime, ' ms'
  );
  Readln;
end.
```

C++

```
#include <windows.h>
#include <iostream>

void main()
{
  // Начальное значение времени
  int StartMs = GetTickCount();

  // Анализируемый фрагмент кода
  ...

  // Конечное значение времени
  int EndMs = GetTickCount();

  // Время выполнения
  std::cout << "Elapsed time: " <<
    EndMs-StartMs << " ms\n";
  std::getchar();
}
```

Основным недостатком использования функций, непосредственно основанных на получении системного времени, является их большая погрешность, составляющая 10–20 мс. Они могут быть использованы только для измерения больших временных интервалов (не менее 1 секунды).

Другим способом измерения времени является использование счетчика TSC (Time Stamp Counter), представляющего собой 64-битный MSR-регистр (Model Specific Register) в составе современных процессоров. В момент включения компьютера его значение

обнуляется, по приходу каждого нового импульса от тактового генератора значение счетчика инкрементируется. Большой диапазон значений счетчика гарантирует отсутствие его переполнения за время работы компьютера (для процессора, работающего на частоте 4 ГГц, время, необходимое для переполнения счетчика, составляет приблизительно 136 лет). Счетчик TSC может быть использован для измерения времени выполнения заданного фрагмента программы с высокой точностью (до 50–100 тактов или 10–20 нс). Для чтения значения счетчика существует команда RDTSC, позволяющая получение значения счетчика в паре регистров EDX:EAX. Данная команда не является привилегированной и допускает чтение счетчика из программы, запущенной на третьем кольце защиты (ring 3). Для чтения значений большинства других MSR-регистров необходимо находиться на нулевом кольце защиты (ring 0), что невозможно из обычной программы и требует разработки специального драйвера.

Пример кода, позволяющий осуществление измерения времени выполнения фрагмента программы с использованием счетчика TSC, приведен ниже:

Delphi

```
program TimeMeasure;
{$APPTYPE Console}

var
  StartTscValue, EndTscValue: Int64;

begin
  // Начальное значение счетчика
  asm
    RDTSC
    MOV     DWORD PTR [StartTscValue], EAX
    MOV     DWORD PTR [StartTscValue+4], EDX
  end;

  // Анализируемый фрагмент кода
  ...

  // Конечное значение счетчика
  asm
    RDTSC
    MOV     DWORD PTR [EndTscValue], EAX
    MOV     DWORD PTR [EndTscValue+4], EDX
  end;

  // Время выполнения
  Writeln('Elapsed time: ',
      EndTscValue-StartValue, ' clocks'
  );
  Readln;
end.
```

C++

```
#include <iostream>

void main()
{
  __int64 StartTscValue, EndTscValue;

  // Начальное значение счетчика
  __asm
  {
    RDTSC
    MOV   DWORD PTR [StartTscValue], EAX
    MOV   DWORD PTR [StartTscValue+4], EDX
  }

  // Анализируемый фрагмент кода
  ...

  // Конечное значение счетчика
  __asm
  {
    RDTSC
    MOV   DWORD PTR [EndTscValue], EAX
    MOV   DWORD PTR [EndTscValue+4], EDX
  }

  // Время выполнения
  std::cout << "Elapsed time: " <<
    EndTscValue - StartTscValue << " clocks\n";
  std::getchar();
}
```

Получение значения счетчика TSC удобно оформить в виде функции. В Delphi соглашение о вызове (calling convention) `register`, используемое по умолчанию, для функции с типом результата `Int64` обеспечивает возврат значения в паре регистров EDX:EAX в 32-битном режиме и в младших половинах регистров RDX:RAX в 64-битном режиме, что не требует добавления дополнительных действий в тело функции. В C++ аналогичным образом можно использовать соглашения о вызове `__cdecl` и `__stdcall`.

Delphi

```
function GetTSC(): Int64; register;
asm
  RDTSC
end;
```

C++

```
__int64 __cdecl GetTSC()
{
  __asm RDTSC
}
```

В современных версиях языка C++ прямое использование команд языка ассемблера или машинного кода не приветствуется, поэтому для многих машинных команд существует высокоуровневые аналоги, называемые интринсиками (intrinsics). Так, например, для команды RDTSC существует функция

```
unsigned __int64 __rdtsc();
```

Вызов intrinsic-функций транслируется компилятором напрямую в целевую команду (в данном случае – RDTSC), функция является inline-подставляемой, дополнительные команды CALL/RET и пр. при ее вызове в ассемблерный код не вставляются, что гарантирует высокую эффективность результирующего машинного кода (в текущей задаче – малую погрешность измерения, вносимую использованием дополнительных ассемблерных команд).

Число тактов, которое занимает выполнение WinAPI-функции `Sleep(1000)` (задержка выполнения потока приблизительно на 1000 мс), приблизительно соответствует текущей тактовой частоте ядра процессора, на котором производится выполнение.

Еще одним способом измерения времени выполнения фрагмента программы является использование WinAPI-функций `QueryPerformanceCounter()` и `QueryPerformanceFrequency()`. Функция `QueryPerformanceCounter()` получает текущее значение высокоточного счетчика, функция `QueryPerformanceFrequency()` возвращает число отсчетов выбранного счетчика в секунду.

Delphi

```
program TimeMeasure;
{$APPTYPE CONSOLE}

uses
  Windows;

var
  StartValue, EndValue, Freq: Int64;
  s: Double;

begin
  // Начальное значение счетчика
  QueryPerformanceCounter(StartValue);

  // Анализируемый фрагмент кода
  ...

  // Конечное значение счетчика
  QueryPerformanceCounter(EndValue);

  // Получание частоты срабатывания счетчика
  QueryPerformanceFrequency(Freq);

  // Расчет времени в секундах
  s := (EndValue - StartValue) / Freq;

  Writeln(EndValue - StartValue, ' ticks');
  Writeln(s:10:10, ' s');

  Readln;
end.
```

C++

```
include <windows.h>
#include <iostream>

void main()
{
  // Начальное значение счетчика
  __int64 StartTicks;
  QueryPerformanceCounter(
    (LARGE_INTEGER *)&StartTicks
  );

  // Анализируемый фрагмент кода
  ...

  // Конечное значение счетчика
  __int64 EndTicks;
  QueryPerformanceCounter(
    (LARGE_INTEGER *)&EndTicks
  );

  // Получение частоты срабатывания счетчика
  __int64 Freq;
  QueryPerformanceFrequency(
    (LARGE_INTEGER *)&Freq
  );

  // Расчет времени в секундах
  double s = (EndTicks - StartTicks) / Freq;

  std::cout << "Elapsed time: \n" <<
    EndTicks-StartTicks << " ticks\n" <<
    s << " s\n";
  std::getchar();
}
```

При выполнении программы на некоторых многоядерных процессорах возможна ситуация, когда значения счетчика TSC для разных ядер различаются. В этом случае если поток начал выполнение на одном ядре в момент времени t_1, а завершил на другом в

момент времени t_2, значение времени выполнения $\Delta t = t_2 - t_1$ может быть некорректным (например, отрицательным). Чтобы измерить время выполнения фрагмента в данном случае, необходимо осуществить жесткую привязку потока к одному из ядер путем вызова WinAPI-функции `SetThreadAffinityMask()`:

```
SetThreadAffinityMask(GetCurrentThread(), 1);
```

Первый параметр функции определяет дескриптор потока, для которого осуществляется привязка (в данном примере, текущий поток); второй – битовую маску ядер, на которых разрешено выполнение потока (например, $1_{10} = 0001_2$ – привязка к первому ядру, $2_{10} = 0010_2$ – ко второму ядру, $5_{10} = 0101_2$ – к первому и третьему ядрам).

Если во время выполнения профилируемого потока управление передается на другой поток (например, возникло прерывание или был запущен другой более приоритетный поток в рамках логики работы современных операционных систем с вытесняющей многозадачностью), то величина суммарного времени выполнения окажется завышенной, т.к. будет включать время выполнения других потоков (погрешность), не имеющих отношения к профилируемому потоку. Для минимизации влияния данной ситуации необходимо либо произвести несколько замеров (если это возможно) с обработкой полученных результатов (например, расчетом среднего арифметического от полученных значений или выбором минимального), либо вручную повысить приоритет профилируемого процесса путем вызова WinAPI-функции `SetPriorityClass()`:

```
SetPriorityClass(GetCurrentProcess(), REALTIME_PRIORITY_CLASS);
```

Первый параметр функции определяет дескриптор процесса, второй – приоритет. Рекомендуемым приоритетом является `HIGH_PRIORITY_CLASS`, превышающим приоритет большинства процессов в системе. В исключительных случаях возможно использование параметра `REALTIME_PRIORITY_CLASS`, однако в данном случае некоторые системные процессы могут не получать достаточного количества квантов процессорного времени, что может приводить, например, к невозможности обработки системой событий от мышки или клавиатуры (в том числе, в некоторых случаях, соответствующих прерываний от периферийных устройств, которыми являются мышь и клавиатура, прерываний от видеокарты и соответствующих им обработчиков в составе драйверов). В случае, если профилируемой фрагмент программы «повиснет», систему придется перезагружать с использованием кнопки Reset на системном блоке. Задержка обработки прерываний от видеокарты может привести к автоматическому перезапуску ее драйвера как повисшего (по мнению операционной системы), смене используемой цветовой схемы на упрощенную и т.д., что далеко не всегда удобно и целесообразно.

При необходимости можно дополнительно поднять приоритет потока, выполняемого в рамках процесса, путем вызова WinAPI-функции `SetThreadPriority()`, однако в большинстве случаев в этом нет необходимости.

Значения времени выполнения одного и того же фрагмента кода, получаемые на различных процессорах, могут отличаться из-за различной микроархитектуры, тактовой частоты, объема кэш-памяти процессора, а также из-за работы функций энергосбережения и динамического управления тактовой частотой (Thermal Throttling, CxE, Turbo Boost, EIST и др.).

Задание. В соответствии с индивидуальным вариантом произвести измерение времени выполнения заданного фрагмента программного кода с использованием перечисленных выше способов. Каждый замер повторить не менее K раз (K задается

пользователем), убедиться в наличии погрешности измерения. Замеры произвести не менее чем на 3 различных процессорах, убедиться в различии получаемых величин.

Пример 1. Измерение малого интервала времени – сортировка массива из $N = 1000$ элементов методом «пузырька», число замеров – $K = 5$.

Анализируемый фрагмент программы:

```
repeat
  Changed := False;
  for I := 1 to N-1 do
    if Arr[I] > Arr[I+1] then begin
      t := Arr[I];
      Arr[I] := Arr[I+1];
      Arr[I+1] := t;
      Changed := True;
    end;
until not Changed;
```

Результаты измерения при помощи `QueryPerformanceFrequency()`:

Замер	1	2	3	4	5
Intel Core 2 Duo E6300 @ 1,86 ГГц	3,578 мс	3,477 мс	3,715 мс	3,899 мс	6,673 мс
AMD Athlon X2 3800+ @ 2,0 ГГц	3,230 мс	3,224 мс	3,173 мс	3,293 мс	3,211 мс
Intel Xeon 5530 @ 2,4 ГГц	3,494 мс	3,540 мс	3,526 мс	3,430 мс	3,491 мс

Результаты почти совпадают на различных процессорах из-за того, что скорость выполнения сортировки определяется скоростью подсистемы памяти (в данном случае, кэш-памяти уровня L1d).

Результаты измерения при помощи счетчика TSC:

Замер	1	2	3	4	5
Intel Core 2 Duo E6300 @ 1,86 ГГц	3,246 мс	3,215 мс	3,168 мс	3,217 мс	3,210 мс
AMD Athlon X2 3800+ @ 2,0 ГГц	4,248 мс	4,908 мс	7,090 мс	3,819 мс	3,459 мс
Intel Xeon 5530 @ 2,4 ГГц	4,472 мс	4,682 мс	4,684 мс	4,577 мс	4,127 мс

Полученные результаты несколько отличаются от цифр предыдущей таблицы.

Результаты измерения при помощи `GetTickCount()`:

Замер	1	2	3	4	5
Intel Core 2 Duo E6300 @ 1,86 ГГц	0,000 мс	0,000 мс	0,000 мс	0,000 мс	0,000 мс
AMD Athlon X2 3800+ @ 2,0 ГГц	0,000 мс	0,000 мс	0,000 мс	0,000 мс	0,000 мс
Intel Xeon 5530 @ 2,4 ГГц	0,000 мс	0,000 мс	0,000 мс	0,000 мс	0,000 мс

Время выполнения фрагмента слишком мало для выбранной функции измерения времени.

Пример 2. Измерение большого интервала времени – факторизация числа 352 700 091 909 229 843 (делится на 7 726 079) , число замеров – $K = 5$.

Анализируемый фрагмент программы:

```
Divisor := -1;
```

```
IsPrime := True;
for I := 2 to Round(sqrt(N)) do
  if N mod I = 0 then begin
    IsPrime := False;
    Divisor := I;
    break;
  end;
```

Результаты измерения при помощи `QueryPerformanceFrequency()`:

Замер	1	2	3	4	5
Intel Core 2 Duo E6300 @ 1,86 ГГц	4,850 с	4,840 с	4,803 с	4,816 с	4,980 с
AMD Athlon X2 3800+ @ 2,0 ГГц	2,846 с	2,867 с	2,867 с	2,843 с	2,913 с
Intel Xeon 5530 @ 2,4 ГГц	4,567 с	4,568 с	4,627 с	4,569 с	4,570 с

Результаты измерения при помощи счетчика TSC:

Замер	1	2	3	4	5
Intel Core 2 Duo E6300 @ 1,86 ГГц	4,677 с	4,737 с	4,784 с	4,775 с	4,664 с
AMD Athlon X2 3800+ @ 2,0 ГГц	3,127 с	3,077 с	3,106 с	3,061 с	3,066 с
Intel Xeon 5530 @ 2,4 ГГц	5,879 с	5,867 с	5,871 с	5,872 с	5,873 с

Результаты измерения при помощи `GetTickCount()`:

Замер	1	2	3	4	5
Intel Core 2 Duo E6300 @ 1,86 ГГц	4,703 с	4,719 с	4,750 с	4,813 с	4,625 с
AMD Athlon X2 3800+ @ 2,0 ГГц	3,016 с	3,000 с	3,000 с	3,015 с	3,016 с
Intel Xeon 5530 @ 2,4 ГГц	4,594 с	4,578 с	4,609 с	4,578 с	4,578 с

Время выполнения приведенного фрагмента кода определяется главным образом временем целочисленного деления (команда `IDIV`). Из результатов измерений видно, что оно различно для различных процессоров.

Более правильной является стратегия автоматического выполнения серии из K измерений с последующей статистической обработкой полученных результатов. Пусть t_i – время i-го замера времени выполнения одного и того же профилируемого фрагмента кода, $T = [t_1, t_2, ..., t_K]$ – выборка результатов измерения. Тогда, в простейшем случае, в качестве результирующих оценок времени исполнения можно выбрать следующие:

- минимальное измеренное время выполнения: $t_{\min}(T) = \min\limits_{i=1,K} t_i$ (в предположении, что «проблема первого прохода», прерывания, вытесняющая многозадачность и т.п. факторы только увеличивают время выполнения профилируемого участка кода);
- среднее арифметическое времен выполнения: $t_{\text{avg}}(T) = \frac{1}{K}\sum\limits_{i=1}^{K} t_i$.

При использовании среднего арифметического (в данном случае, средневыборочного значения) желательно производить оценку ширины доверительного интервала, в рамках которого располагается искомое мат. ожидание t_{mean} для заданной доверительной вероятности p, и отбрасывать значения, сильно выбивающиеся из общего ряда (например, с использованием критерия грубого выброса – в простейшем случае, оставление данных, отличающихся от среднего не более чем на 3σ). Для этого необходимо:

1. Рассчитать средневыборочное значение t_{avg} (см. формулу выше).
2. Рассчитать дисперсию: $D(T)=\dfrac{(t_i-t_{\text{avg}})^2}{K}$.
3. Рассчитать среднеквадратическое отклонение (СКО): $\sigma(T)=\sqrt{D(T)}$.
4. Отбросить значения t_j, отличающиеся от среднего более чем на 3σ: $\forall t_j, |t_j-t_{\text{avg}}|>3\sigma(T)$: $T':=T\setminus\{t_j\}$.
5. Пересчитать значения среднего арифметического $t_{avg}(T')$, дисперсии $D(T')$ и СКО $\sigma(T')$ для полученной выборки T'.
6. Оценить ширину доверительного интервала $\Delta t=\dfrac{1{,}96\sigma(T')}{\sqrt{K}}$, где 1,96 – квантиль (процентиль) распределения Стьюдента для доверительной вероятности $p=0{,}95$.
7. Оценить доверительный интервал $[t_{\text{avg}}(T')-\Delta t; t_{\text{avg}}(T')+\Delta t]$, результирующее значение времени записать в виде $t_{\text{avg}}(T')\pm\Delta t$.

Указанные выше формулы справедливы только в том случае, если распределение искомой величины t является нормальным. В общем случае на практике это не так: распределение может иметь эксцесс, быть усеченным, мультимодальным и т.п. В данном случае для выполнения более корректной оценки времени необходимо:

1. Построить гистограмму распределения величины t путем разбиения интервала ее изменения $[t_{\min}; t_{\max}]$ на M отрезков (M задается). В случае наличия нескольких мод в распределении оставить только «меньшую» («младшую») моду путем исключения из выборки T значений, принадлежащих к другим («старшим») модам с получением выборки T''.
2. Рассчитать средневыборочное значение $t_{\text{avg}}(T'')$.
3. Отсортировать выборку: $T'':=\text{sort}(T'')$.
4. Рассчитать ширину доверительного интервала в элементах: $I=|T''|\cdot p$, где $|T''|$ – мощность (число элементов) в выборке T'', p – выбранная доверительная вероятность (например, 0,95).
5. Выбрать такие индексы начального i_{start} и конечного $i_{end}=i_{start}+I-1$ элементов выборки $t_{i_{start}}\in T''$ и $t_{i_{end}}\in T''$, что ширина доверительного интервала минимальна: $d(i_{start}, i_{end})=t_{i_{end}}-t_{i_{start}}\to\min$.
6. Записать результирующий доверительный интервал в виде $[t_{i_{start}}; t_{i_{end}}]$.
7. Рассчитать среднее значение как $t_{avg}(T''')=\dfrac{1}{I}\sum\limits_{i=i_{start}}^{i_{end}} t_i$.

Сравнить полученные значения времен $t_{\min}$, $t_{\text{avg}}(T)$, $t_{\text{avg}}(T')$, $t_{\text{avg}}(T'')$, $t_{\text{avg}}(T''')$ и соответствующие им доверительные интервалы (для последних четырех величин).

Пример.

Замер (t_i)	1	2	3	4	5
Intel Core 2 Duo E6300 @ 1,86 ГГц	3,578 мс	3,477 мс	3,715 мс	3,899 мс	6,673 мс

Минимальное измеренное время выполнения:

$t_{\min} = 3{,}477$ мс.

Среднее время (без статистической обработки):

$T = [3{,}578;\ 3{,}477;\ 3{,}715;\ 3{,}899;\ 6{,}673]$,

$$t_{\text{avg}}(T) = \frac{3{,}578 + 3{,}477 + 3{,}715 + 3{,}899 + 6{,}673}{5} = 4{,}268.$$

(среднее значение завышено из-за статистического выброса $t_5 = 6{,}673$)

$D(T) = 1{,}466$,

$\sigma(T) = \sqrt{1{,}466} = 1{,}211$,

$$\Delta t = \frac{1{,}96 \cdot 1{,}211}{\sqrt{5}} = 1{,}061.$$

Среднее время с учетом погрешности измерения: $t_{\text{avg}}(T) = 4{,}268 \pm 1{,}061$ мс.

Доверительный интервал: $[3{,}207; 5{,}329]$.

Среднее время (отброс значения t_5 по критерию грубого выброса):

$T' = [3{,}578;\ 3{,}477;\ 3{,}715;\ 3{,}899;\ 6{,}673] \setminus \{t_5 = 6{,}673\} = [3{,}578;\ 3{,}477;\ 3{,}715;\ 3{,}899]$,

$$t_{\text{avg}}(T') = \frac{3{,}578 + 3{,}477 + 3{,}715 + 3{,}899}{4} = 3{,}667.$$

(среднее значение существенно снизилось из-за ликвидации грубого выброса)

$D(T') = 0{,}025$,

$\sigma(T') = \sqrt{0{,}025} = 0{,}158$,

$$\Delta t = \frac{1{,}96 \cdot 0{,}158}{\sqrt{4}} = 0{,}155.$$

Среднее время с учетом погрешности измерения: $t_{\text{avg}}(T') = 3{,}667 \pm 0{,}155$ мс.

Доверительный интервал: $[3{,}512; 3{,}822]$.

Среднее время (произвольное распределение, не являющееся нормальным):

Пример гистограмм (на рисунках ниже гистограммы построена для $K = 10\,000$ измерений для большей наглядности, число интервалов – 100):

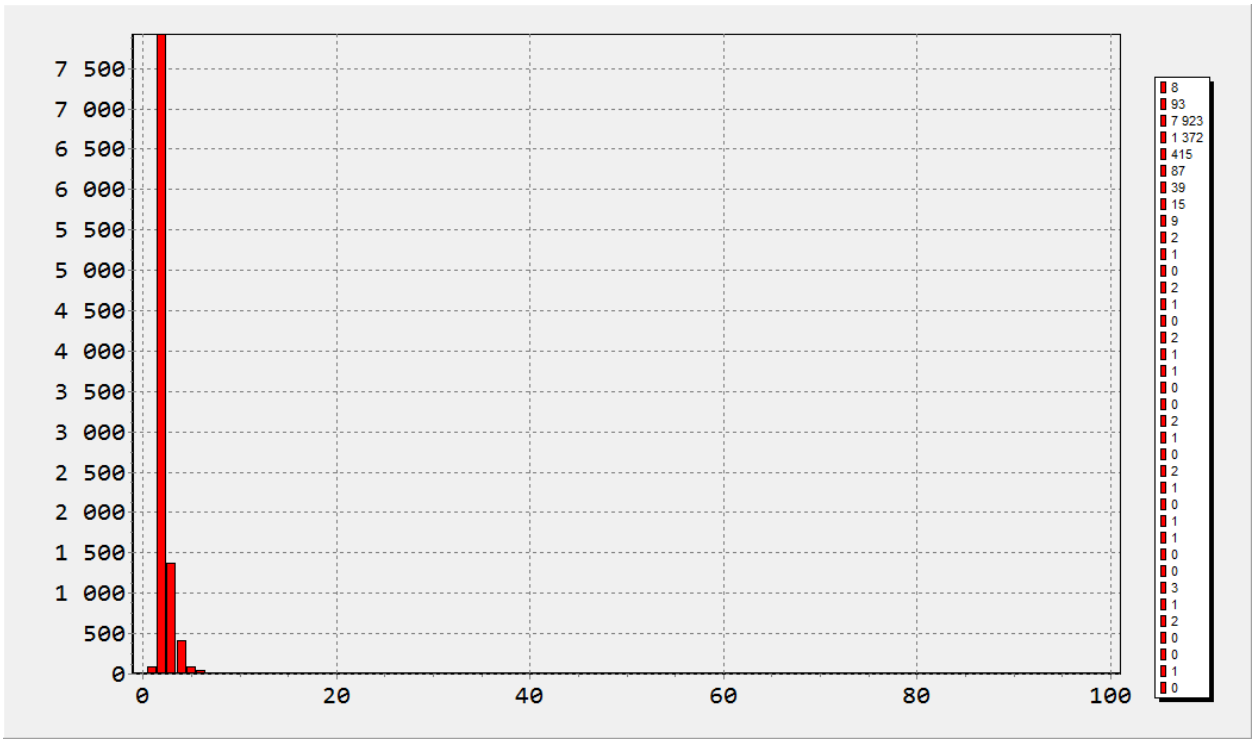

Рис. 1. Пример гистограммы без отсева грубых выбросов

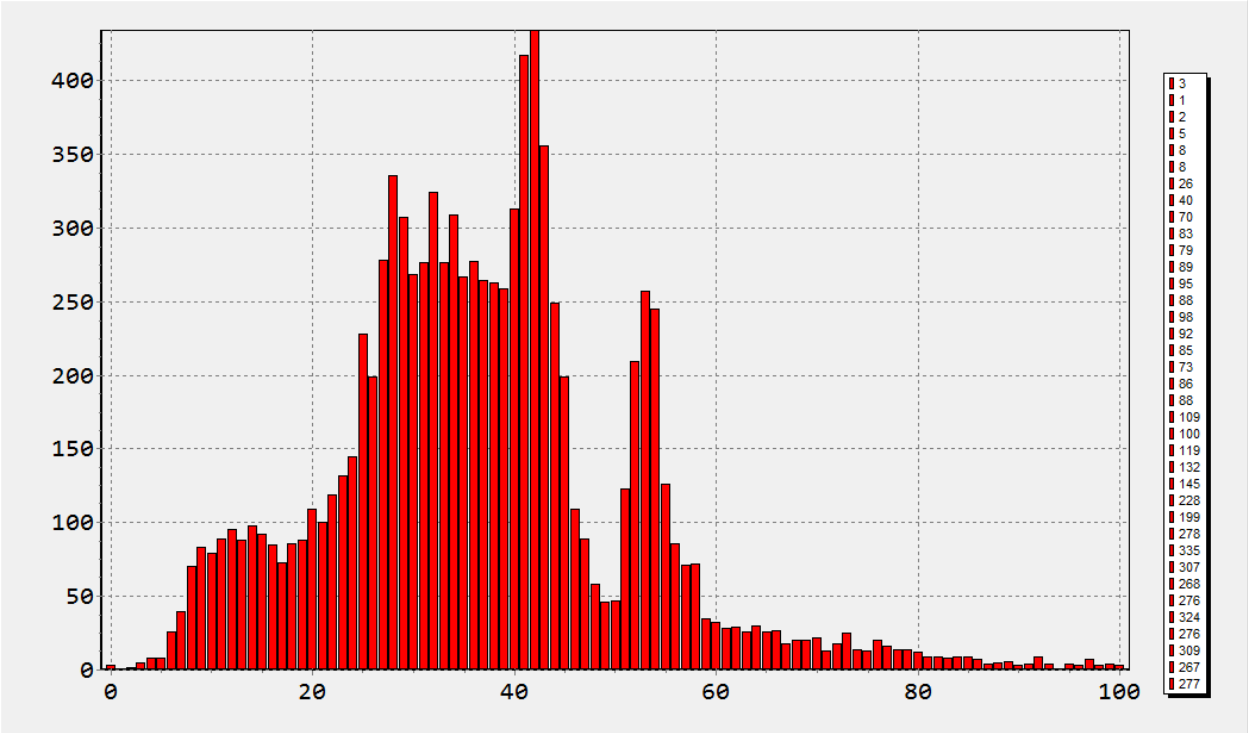

Рис. 2. Пример гистограммы с отсевом грубых выбросов (пример мультимодального распределения)

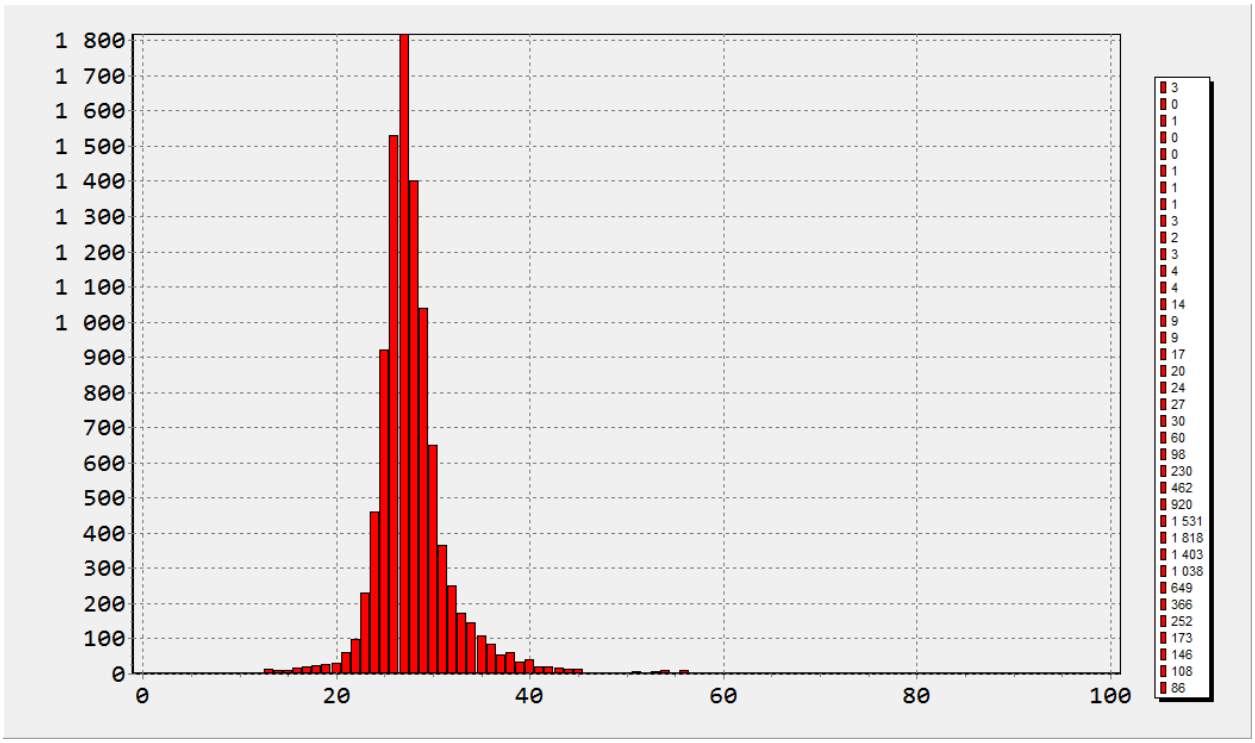

Рис. 3. Пример гистограммы с отсевом грубых выбросов (пример унимодального распределения, близкого к нормальному, с незначительным эксцессом)

$t_{\min} = 3{,}477,$

$t_{\max} = 6{,}673.$

Исключение мод не производилось,

$T'' = T = [3{,}578;\ 3{,}477;\ 3{,}715;\ 3{,}899;\ 6{,}673],$

$sort(T'') = [3{,}477;\ 3{,}578;\ 3{,}715;\ 3{,}899;\ 6{,}673].$

$p = 0{,}6.$

(с учетом малого объема выборки $K = 5$, взятого исключительно в иллюстративных целях, выбрано достаточно малое значение доверительной вероятности; при выполнении лабораторной работы выбирать $p = 0{,}95$ аналогично рассмотренному в примере выше)

$I = |T''| \cdot p = 5 \cdot 0{,}6 = 3.$

Выбор границ доверительного интервала:

$i_{start} = 1,\ i_{end} = 1 + 3 - 1 = 3,\ d(1, 3) = 3{,}715 - 3{,}477 = 0{,}238$ (min!);

$i_{start} = 2,\ i_{end} = 2 + 3 - 1 = 4,\ d(2, 4) = 3{,}899 - 3{,}578 = 0{,}321;$

$i_{start} = 3,\ i_{end} = 3 + 3 - 1 = 5,\ d(3, 5) = 6{,}673 - 3{,}715 = 2{,}958.$

Доверительный интервал: $[3{,}477; 3{,}715]$.

Среднее время для выбранного доверительного интервала:

$$t_{\text{avg}}(T''') = \frac{3{,}477 + 3{,}578 + 3{,}715}{3} = 3{,}590 \text{ мс.}$$

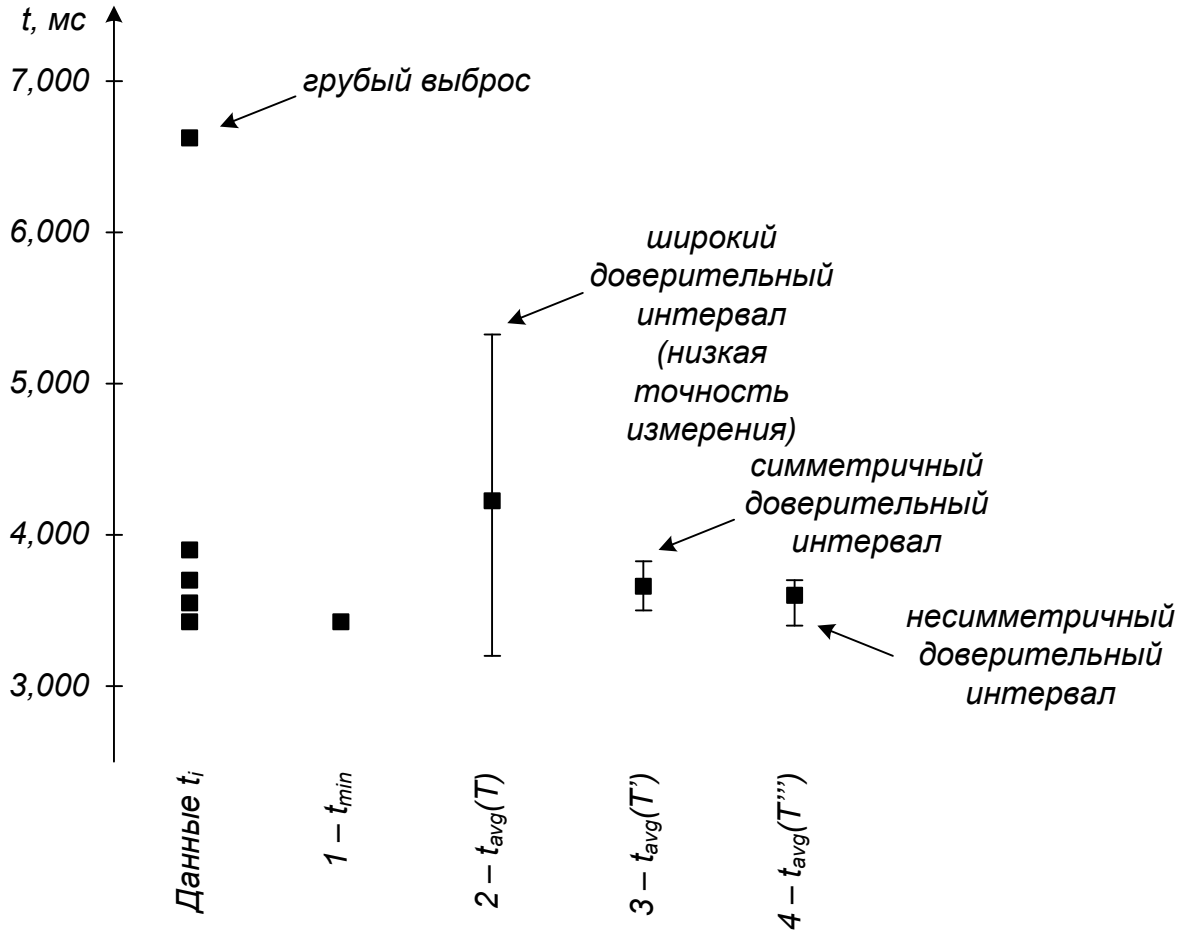

Рис. 4. Результаты измерения в графическом виде

Содержание отчета.

1. Титульный лист.
2. Цель работы.
3. Описание условия решаемой задачи в соответствии с индивидуальным вариантом.
4. Листинг программы.
5. Результаты измерения времени выполнения фрагмента программы всеми возможными способами (гистограммы, минимальное время, среднее время и доверительные интервалы для каждого из способов).
6. Выводы.